# TP1 — Réseaux Linux : espaces de noms, veth, bridge et NAT

**Module :** Réseaux : cloud et Internet des objets — Master 2 RSA

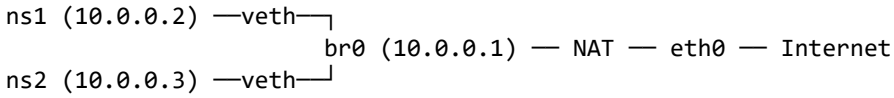
## Savoir-faire

- Créer des espaces de noms réseau et les relier par des paires veth.
- Construire un commutateur virtuel (bridge) et lui connecter plusieurs espaces de noms.
- Donner un accès à Internet à des réseaux privés par routage et NAT.
- Observer et analyser le trafic avec `tcpdump`.
- Retrouver ces mécanismes dans le réseau d'un conteneur Docker.

## Notions

### Introduction

Quand on virtualise, on crée des machines (virtuelles ou conteneurs) qui doivent communiquer entre elles et avec l'extérieur. Le noyau Linux fournit des briques simples, que les hyperviseurs, Docker et les plateformes cloud assemblent à grande échelle. Ce TP vous les fait construire à la main.




### Les briques

| Brique | Rôle | Équivalent physique |
|---|---|---|
| **Espace de noms réseau** (*network namespace*) | Pile réseau isolée : interfaces, adresses, routes, règles de filtrage propres | Une machine |
| **Paire veth** | Deux interfaces virtuelles reliées : ce qui entre par l'une sort par l'autre | Un câble réseau |
| **Bridge** | Commutateur de niveau 2 : il apprend les adresses MAC et relaie les trames | Un switch |
| **Routage** (`ip_forward`) | Le noyau transmet les paquets d'une interface à une autre | Un routeur |
| **NAT** (`MASQUERADE`) | Remplace l'adresse source privée par l'adresse de la sortie | Une box Internet |

### Obtenir de l'aide

Comme pour la commande `docker`, l'aide est disponible à chaque niveau :

```
ip help
ip link help
ip netns help
man ip-netns
```

Beaucoup de commandes de ce TP doivent être lancées avec les droits administrateur (`sudo`).

## Environnement de travail

- Un Linux avec `sudo` : machine virtuelle Ubuntu ou Debian (VirtualBox, VMware Workstation), WSL2.
- Les outils `ip` (paquet `iproute2`), `ping`, `iptables` et `tcpdump` :

  ```
  sudo apt install -y iproute2 iputils-ping iptables tcpdump
  ```

  `ping` est absent des images Debian et Ubuntu minimales : ne sautez pas `iputils-ping`.
- Deux terminaux ouverts : un pour les commandes, un pour observer le trafic.

Vérifiez votre environnement, puis relevez votre **interface de sortie** et mémorisez-la dans une variable `IFACE`, utilisée dans toute la suite :

```
ip -V
sudo -v
ip route show default
# exemple de résultat : default via 192.168.1.1 dev ens33

IFACE=ens33        # à adapter : recopiez le nom affiché par la commande ci-dessus
echo "$IFACE"
```

> **Attention :** cette variable n'existe que dans le terminal où vous l'avez définie. Dans votre second terminal, et à chaque nouvelle session, redéfinissez-la avant toute commande qui l'utilise. Un `echo "$IFACE"` qui ne renvoie rien signale que la variable est vide : les commandes `iptables` échoueraient alors sur une erreur de syntaxe difficile à interpréter.
>
> Les commandes de ce TP modifient la configuration réseau de la machine. Travaillez sur une machine virtuelle ou de test et exécutez le nettoyage final.

**Compte rendu.** Conservez vos commandes et les captures des résultats importants : elles vous serviront pour le schéma d'architecture réseau de votre projet.

---

## Exercice 1 : Espaces de noms et paire veth

**Objectif :** créer deux « machines » isolées et les relier par un câble virtuel.

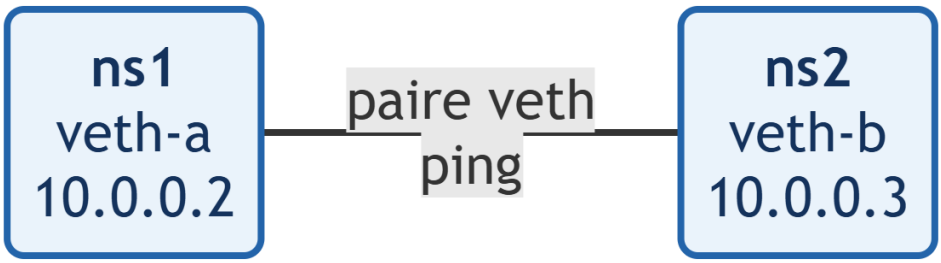



FIGURE 1 – ns1 (veth-a, 10.0.0.2) et ns2 (veth-b, 10.0.0.3) reliés par une paire veth

*Vous créez deux espaces de noms, leur attribuez une adresse IP et vérifiez qu'ils communiquent.*

1. Créez deux espaces de noms et listez-les :

   ```
   sudo ip netns add ns1
   sudo ip netns add ns2
   ip netns list
   ```

2. Créez une paire veth et placez une extrémité dans chaque espace de noms :

   ```
   sudo ip link add veth-a type veth peer name veth-b
   sudo ip link set veth-a netns ns1
   sudo ip link set veth-b netns ns2
   ```

3. Configurez les adresses et activez les interfaces :

   ```
   sudo ip netns exec ns1 ip addr add 10.0.0.2/24 dev veth-a
   sudo ip netns exec ns1 ip link set veth-a up
   sudo ip netns exec ns1 ip link set lo up
   sudo ip netns exec ns2 ip addr add 10.0.0.3/24 dev veth-b
   ```

```
sudo ip netns exec ns2 ip link set veth-b up
sudo ip netns exec ns2 ip link set lo up
```

4. Testez la communication :

```
sudo ip netns exec ns1 ping -c 3 10.0.0.3
```

**Questions**

1. Exécutez `ip addr` sur l'hôte, puis dans `ns1` (`sudo ip netns exec ns1 ip addr`). Que constatez-vous ?
2. Voyez-vous `veth-a` dans le résultat de `ip link` sur l'hôte ? Pourquoi ?
3. Relevez l'adresse MAC de `veth-a` et de `veth-b`. Dans `ns1`, que contient la table de voisins (`ip neigh`) après le ping ?
4. Lancez `sudo ip netns exec ns1 ip route`. D'où vient la route affichée ?
5. Essayez `sudo ip netns exec ns1 ping -c 1 8.8.8.8`. Que se passe-t-il, et pourquoi ?

**Ce qu'il faut comprendre :** un espace de noms ne connaît que ses propres interfaces. Tant qu'on n'a pas relié ses interfaces au reste du réseau, il n'a aucun moyen d'en sortir.

Avant l'exercice 2, supprimez la paire. Les deux extrémités sont maintenant dans les espaces de noms, pas sur l'hôte : la commande doit donc être lancée depuis `ns1`. Supprimer une extrémité supprime l'autre.

```
sudo ip netns exec ns1 ip link del veth-a
```

Vérifiez que les adresses ont bien disparu, sinon l'exercice 2 refusera de réattribuer `10.0.0.2/24` :

```
sudo ip netns exec ns1 ip addr
sudo ip netns exec ns2 ip addr
```

---

## Exercice 2 : Un commutateur virtuel (bridge)

**Objectif :** relier plusieurs espaces de noms comme avec un switch.

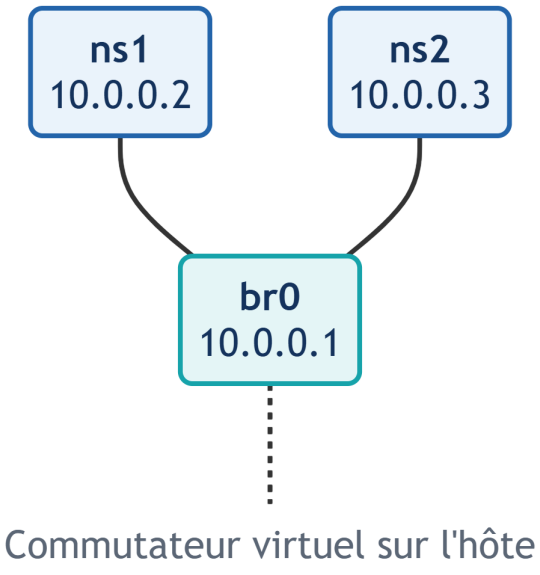



FIGURE 2 – ns1 (10.0.0.2) et ns2 (10.0.0.3) raccordés au bridge br0 (10.0.0.1), commutateur virtuel porté par l'hôte

*Les deux espaces de noms ne se voient plus par un câble direct, mais à travers un commutateur porté par l'hôte.*

1. Créez le bridge et donnez-lui une adresse, qui servira de passerelle :

```
sudo ip link add br0 type bridge
sudo ip addr add 10.0.0.1/24 dev br0
sudo ip link set br0 up
```

2. Pour chaque espace de noms, créez une paire veth dont une extrémité va dans l'espace et l'autre sur le bridge. Pour `ns1` :

```
sudo ip link add veth1 type veth peer name veth1-br
sudo ip link set veth1 netns ns1
sudo ip link set veth1-br master br0
sudo ip link set veth1-br up
sudo ip netns exec ns1 ip addr add 10.0.0.2/24 dev veth1
sudo ip netns exec ns1 ip link set veth1 up
sudo ip netns exec ns1 ip link set lo up
```

   Répétez avec `veth2`, `veth2-br`, `ns2` et l'adresse `10.0.0.3/24`.

3. Testez :

```
sudo ip netns exec ns1 ping -c 3 10.0.0.3
sudo ip netns exec ns1 ping -c 3 10.0.0.1
bridge link
```

   **Si le ping échoue et que Docker est installé sur la machine :** Docker insère ses propres règles de filtrage, qui peuvent bloquer le trafic traversant votre bridge. Commencez par regarder ce qu'il en est :

```
sudo iptables -S FORWARD | head -4
```

   Jusqu'à Docker 27, la politique de la chaîne devenait `-P FORWARD DROP`. Depuis Docker 28, elle reste `ACCEPT` et le filtrage passe par les chaînes `DOCKER-USER` et `DOCKER-FORWARD`. Dans les deux cas, autorisez explicitement votre bridge :

```
sudo iptables -I FORWARD -i br0 -o br0 -j ACCEPT
```

**Questions**

1. Quelles interfaces `bridge link` affiche-t-il ? Quel est leur état ?
2. Pourquoi `ns1` peut-il joindre `10.0.0.1`, adresse portée par l'hôte ?
3. Affichez la table d'adresses MAC apprises par le bridge : `bridge fdb show br br0`. Que contient-elle ?
4. Pendant un ping de `ns1` vers `ns2`, lancez dans l'autre terminal `sudo tcpdump -ni br0`. Que voyez-vous ? Voyez-vous d'abord des requêtes ARP ?

**Ce qu'il faut comprendre :** le bridge se comporte comme un switch : il apprend les adresses MAC des interfaces connectées et relaie les trames entre elles. L'adresse IP posée sur `br0` donne à l'hôte une présence sur ce réseau virtuel.

---

## Exercice 3 : Routage et NAT

**Objectif :** donner un accès à Internet à des réseaux privés.

Les espaces de noms ont des adresses privées. Pour sortir, l'hôte doit router leurs paquets et traduire leurs adresses (NAT).

1. Indiquez à chaque espace de noms que la passerelle est le bridge :

```
sudo ip netns exec ns1 ip route add default via 10.0.0.1
sudo ip netns exec ns2 ip route add default via 10.0.0.1
```

2. Activez le routage sur l'hôte et la traduction d'adresses en utilisant la variable `IFACE` définie plus haut :

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo iptables -t nat -A POSTROUTING -s 10.0.0.0/24 -o "$IFACE" -j MASQUERADE
```

Si votre pare-feu n'autorise pas le transit par défaut — politique FORWARD à DROP, ou règles insérées par Docker (voir l'encadré de l'exercice 2) :

```
sudo iptables -A FORWARD -i br0 -o "$IFACE" -j ACCEPT
sudo iptables -A FORWARD -i "$IFACE" -o br0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
```

3. Dans le second terminal, observez le trafic des deux côtés du NAT :

```
sudo tcpdump -ni br0 icmp
sudo tcpdump -ni "$IFACE" icmp
```

4. Depuis le premier terminal, testez avec une adresse IP, pour ne pas dépendre de la résolution de noms :

```
sudo ip netns exec ns1 ping -c 3 8.8.8.8
```

**Questions**

1. Quelle adresse source voit-on sur `br0` ? Sur `$IFACE` ? Que concluez-vous sur le rôle du NAT ?
2. Que se passe-t-il pour le ping si vous remettez `net.ipv4.ip_forward` à 0 ? Si vous retirez la règle MASQUERADE ?
3. Affichez les règles NAT : `sudo iptables -t nat -L -n -v`. Que montre le compteur de la règle MASQUERADE ?
4. Essayez maintenant un ping vers un nom : `sudo ip netns exec ns1 ping -c 1 example.org`. Selon la configuration de votre machine, il aboutit ou il échoue. Comparez les deux fichiers de résolution :

```
cat /etc/resolv.conf
sudo ip netns exec ns1 cat /etc/resolv.conf
```

Si l'hôte utilise un résolveur local (`nameserver 127.0.0.53`, cas d'Ubuntu avec `systemd-resolved`), pourquoi cette adresse n'est-elle pas joignable depuis `ns1` ? Si l'hôte utilise un résolveur public, pourquoi le ping par nom fonctionne-t-il ? *(indice : `ip netns exec` monte `/etc/netns/<nom>/resolv.conf` par-dessus `/etc/resolv.conf` s'il existe, sinon il garde celui de l'hôte.)* Proposez une correction avec `/etc/netns/ns1/resolv.conf`.

**Ce qu'il faut comprendre :** la machine hôte joue le rôle d'un routeur avec NAT. Aucun paquet d'une adresse privée ne sort tel quel sur Internet : il est réécrit à la sortie, et la réponse est rétablie grâce à la table de suivi de connexions.

> **Astuce :** pour voir les connexions suivies par le NAT : `sudo conntrack -L` (paquet `conntrack`, à installer si besoin).

---

## Exercice 4 : Segmenter le réseau (pour aller plus loin)

**Objectif :** séparer deux réseaux et décider qui peut parler à qui, comme avec deux VLAN reliés par un routeur.

> **Prérequis :** cet exercice suppose que le routage est activé sur l'hôte, ce que vous avez fait à l'exercice 3. Si vous avez sauté l'exercice 3, activez-le maintenant, sinon aucun paquet ne passera d'un réseau à l'autre : `sudo sysctl -w net.ipv4.ip_forward=1`

1. Créez un second bridge et un troisième espace de noms sur un autre sous-réseau :

```
sudo ip netns add ns3
sudo ip link add br1 type bridge
sudo ip addr add 10.0.1.1/24 dev br1
sudo ip link set br1 up
sudo ip link add veth3 type veth peer name veth3-br
```

```
sudo ip link set veth3 netns ns3
sudo ip link set veth3-br master br1
sudo ip link set veth3-br up
sudo ip netns exec ns3 ip addr add 10.0.1.2/24 dev veth3
sudo ip netns exec ns3 ip link set veth3 up
sudo ip netns exec ns3 ip link set lo up
sudo ip netns exec ns3 ip route add default via 10.0.1.1
```

2. Testez la communication entre `ns1` (réseau `10.0.0.0/24`) et `ns3` (réseau `10.0.1.0/24`) :

```
sudo ip netns exec ns1 ping -c 3 10.0.1.2
```

   Si le ping échoue et que Docker est installé, autorisez le passage entre les deux bridges : `sudo iptables -I FORWARD -i br0 -o br1 -j ACCEPT` et `sudo iptables -I FORWARD -i br1 -o br0 -j ACCEPT`

3. Interdisez à `ns3` d'**ouvrir** des connexions vers `ns1`, avec le pare-feu de l'hôte, puis vérifiez dans les deux sens :

```
sudo iptables -I FORWARD -s 10.0.1.0/24 -d 10.0.0.0/24 -m conntrack --ctstate NEW -j DROP
sudo ip netns exec ns3 ping -c 3 -W 1 10.0.0.2
sudo ip netns exec ns1 ping -c 3 -W 1 10.0.1.2
```

**Questions**

1. Pourquoi `ns1` joint-il `ns3` alors qu'ils sont sur deux bridges différents ? Qu'est-ce qui joue le rôle de routeur entre les deux réseaux ?
2. Après la règle, quel ping réussit et lequel échoue ? Pourquoi la règle porte-t-elle sur l'état `NEW` ? Que se passerait-il sans `-m conntrack --ctstate NEW` ?
3. Quel équivalent de cette segmentation retrouve-t-on dans un VLAN et dans une DMZ ?

**Ce qu'il faut comprendre :** deux réseaux sont isolés tant qu'aucun routeur ne les relie ; le pare-feu décide ensuite des flux autorisés. C'est le principe de la segmentation réseau, que vous utiliserez pour isoler les services de votre projet.

---

## Exercice 5 : Les mêmes briques dans Docker

**Objectif :** retrouver dans Docker ce que vous venez de construire.

Docker est installé ? Sinon : `sudo apt install -y docker.io`.

1. Listez les interfaces réseau et localisez l'interface `docker0` :

```
ip a
```

   — Quelle est l'adresse IP associée à `docker0` ? Quelle est son adresse MAC ?
   — Quelle est l'adresse réseau (préfixe et masque) ? Comparez avec `br0` de l'exercice 2.

2. Observez le réseau Docker par défaut :

```
sudo docker network ls
sudo docker network inspect bridge
```

   Relevez le sous-réseau (`Subnet`) et la passerelle (`Gateway`).

3. Lancez deux conteneurs et observez les interfaces créées :

```
sudo docker run -d --name c1 alpine sleep 1000
sudo docker run -d --name c2 alpine sleep 1000
ip a
bridge link
sudo docker exec c1 ip a
```

Quelles interfaces `veth` sont apparues sur l'hôte ? À quelle interface sont-elles reliées ? Repérez l'indice commun entre `eth0@ifN` dans le conteneur et l'interface `veth` sur l'hôte.

4. Testez la communication entre conteneurs :

```
sudo docker exec c1 ping -c 2 c2
sudo docker exec c1 ping -c 2 <adresse IP de c2>
```

Le ping par nom fonctionne-t-il ? Et par adresse IP ? Pourquoi, d'après vous ?

5. Créez un réseau personnalisé et recommencez :

```
sudo docker network create reseau1
sudo docker run -d --network reseau1 --name c3 alpine sleep 1000
sudo docker run -d --network reseau1 --name c4 alpine sleep 1000
sudo docker exec c3 ping -c 2 c4
```

Que changent les réseaux définis par l'utilisateur ? Un conteneur de `reseau1` peut-il joindre `c1` ?

6. Retrouvez l'espace de noms d'un conteneur depuis l'hôte :

```
sudo docker inspect -f '{{.State.Pid}}' c1
sudo ls -l /proc/<PID>/ns
sudo nsenter -t <PID> -n ip a
```

Quel lien faites-vous avec l'exercice 1 ?

7. Retrouvez la règle de NAT utilisée par Docker :

```
sudo iptables -t nat -S POSTROUTING
```

**Ce qu'il faut comprendre :** Docker n'invente rien : chaque conteneur a son espace de noms réseau, relié par une paire veth au bridge `docker0`, avec un NAT pour sortir. Les réseaux définis par l'utilisateur ajoutent la résolution de noms entre conteneurs.

Supprimez les conteneurs et le réseau de test :

```
sudo docker rm -f c1 c2 c3 c4
sudo docker network rm reseau1
```

> **Attention aux suppressions en masse.** Le filtre `name=` de Docker cherche le motif **n'importe où** dans le nom : `-f "name=c"` sélectionne aussi bien `c1` qu'un conteneur de projet appelé `mon-backend`, parce que son nom contient un `c`. Si vous tenez à supprimer en une fois, ancrez le motif et **affichez toujours la liste avant** :
>
> ```
> sudo docker container ls -a -f "name=^c[1-4]$" --format '{{.Names}}'
> sudo docker rm -f $(sudo docker container ls -a -q -f "name=^c[1-4]$")
> ```

---

## Votre synthèse

Remplissez le tableau ci-dessous et conservez-le.

| Action | Commande |
|---|---|
| Créer un espace de noms | |
| Lister les espaces de noms | |
| Exécuter une commande dans un espace de noms | |
| Créer une paire veth | |
| Placer une interface dans un espace de noms | |
| Créer un bridge | |
| Rattacher une interface à un bridge | |

| Action | Commande |
|---|---|
| Affecter une adresse IP à une interface | |
| Activer une interface | |
| Ajouter une route par défaut | |
| Activer le routage IPv4 | |
| Ajouter une règle de NAT (masquerade) | |
| Observer le trafic d'une interface | |
| Lister les réseaux Docker | |
| Créer un réseau Docker | |
| Afficher le détail d'un réseau Docker | |

---

## Nettoyage

Espaces de noms et bridges :

```
sudo ip netns del ns1
sudo ip netns del ns2
sudo ip netns del ns3
sudo ip link del br0
sudo ip link del br1
```

Règle de NAT de l'exercice 3 :

```
sudo iptables -t nat -D POSTROUTING -s 10.0.0.0/24 -o "$IFACE" -j MASQUERADE
```

Règles de filtrage. Supprimez celles que vous avez réellement ajoutées : la règle de segmentation de l'exercice 4, et, si votre pare-feu l'exigeait, les autorisations de passage des exercices 2, 3 et 4.

```
sudo iptables -D FORWARD -s 10.0.1.0/24 -d 10.0.0.0/24 -m conntrack --ctstate NEW -j DROP
sudo iptables -D FORWARD -i br0 -o br0 -j ACCEPT
sudo iptables -D FORWARD -i br0 -o "$IFACE" -j ACCEPT
sudo iptables -D FORWARD -i "$IFACE" -o br0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
sudo iptables -D FORWARD -i br0 -o br1 -j ACCEPT
sudo iptables -D FORWARD -i br1 -o br0 -j ACCEPT
```

Vérifiez qu'il ne reste rien de votre passage :

```
ip netns list
ip -br link show type bridge
sudo iptables -t nat -S POSTROUTING
sudo iptables -S FORWARD
```